PyCalc User's Guide

By Wyatt Fullerton

What PyCalc does

PyCalc is a menu-driven Python calculator with 14 operations. It runs once per launch: pick an operation, enter your numbers, read the answer, and the program ends. To do another calculation, run it again.

Running it

- Run `main.py` with `python3 main.py` (on Windows: `python main.py`), or paste it into the Programiz online compiler(or any other) and press Run.
- You need Python 3.8 or newer. No extra packages are needed.

The menu

When PyCalc starts it asks:

`Enter Operation(sum=1, multiply=2, divide=3, ... ratio simplifier=13, fullen and adden numbers=14):`

Type a number from 1 to 14 and press Enter. Anything outside that range ends the program with "Unknown operator".

All 14 operations at a glance

#	Operation	What it asks for	Example
1	Sum	1st number, 2nd number	4 and 3 gives `7.0`
2	Multiply	1st number, 2nd number	4 and 3 gives `12.0`
3	Divide	1st number, 2nd number	10 and 4 gives `2.50`
4	Difference	1st number, 2nd number	10 and 4 gives `6.0` (1st minus 2nd)

#	Operation	What it asks for	Example
5	Power	base, then exponent	2 and 3 gives `8.0`
6	Mean	how many data points, then each one	2, 4, 9 gives `5.0`
7	Divide with remainder	1st number, 2nd number	17 and 5 gives `3r2`
8	Factorial	one whole number	5 gives `120`
9	Exponential growth	starting number, modifier, repeat count	see below
10	Scientific conversion	one number	`123456789` gives `1.23457e+08`
11	Percent of	the percent, then the amount	20 and 50 gives `10.0`
12	Converter	mode (1 to 8), then the amount	see below
13	Ratio simplifier	1st number, 2nd number	6 and 4 gives `3:2`
14	Fullen and Adden numbers	a whole number, then a mode (1 or 2)	5 and mode 2 gives 35

Operation details

3. Divide

Shows the answer rounded to 2 decimal places, then asks `More?`. Type `T` or `t` to also see the full-precision answer. Anything else finishes.

7. Divide with remainder

Gives the whole-number quotient, the letter `r`, then the remainder. 17 and 5 gives `3r2`.

8. Factorial

Whole numbers only, and no negatives. A decimal like `2.5` causes an error. Very large inputs (like 100000) take a long time.

9. Exponential growth

Applies the same operation to a number over and over, printing every step. It asks three questions:

1. **Starting number:** for example `2`
2. **Operation:** an operator and a number, like `* 3`. Type it without brackets. The `[* 7]` in the prompt is just an example. The space is optional, so `*3` and `**2` also work.
3. **How many times:** for example `4`

Result for `2`, `* 3`, `4`:

```
2.0
6.0
18.0
54.0
162.0
```

Rules for the modifier:

- Allowed operators: `+`, `-`, `*`, `/`, `**`
- No letters and no brackets
- 10 characters or fewer

Numbers grow fast with `**`. Starting at 200 with `**2`, the values pass 10^300 after about 8 repeats and the program stops with an overflow error.

10. Scientific conversion

Rounds a number to 6 significant digits and switches to scientific notation for very large or very small numbers.

You type	You get
`123456789`	`1.23457e+08`
`0.00001234`	`1.234e-05`
`500`	`500`

11. Percent of

The percent goes first, then the amount. 20 and 50 means "20% of 50" and gives `10.0`.

12. Converter

Pick a mode, then type the amount:

Mode	Converts	Example
1	miles to km	1 gives `1.609344`
2	km to miles	1.609344 gives about `1.0`
3	Celsius to Fahrenheit	100 gives `212.0`
4	Fahrenheit to Celsius	212 gives `100.0`
5	pounds to kg	10 gives about `4.536`
6	kg to pounds	10 gives about `22.05`
7	fluid ounces to ml	1 gives `29.57353`
8	ml to fluid ounces	100 gives about `3.381`

Any mode outside 1 to 8 ends with an "Unknown conversion" error.

13. Ratio simplifier

Reduces two numbers to their simplest whole-number ratio.

You type	You get
6 and 4	`3:2`
12 and 18	`2:3`
2.5 and 1.5	`5:3`
-6 and 4	`-3:2`
0 and 5	`0:1`

Decimals work too. Both numbers are scaled up to whole numbers before simplifying. Entering 0 and 0 gives an error because there is nothing to simplify.

14. Fullen and Adden numbers

Two number sequences. PyCalc first asks `What number?`, then the mode: 1 for Fullen, 2 for Adden. Enter whole numbers for both.

- **Fullen (mode 1):** adds up every whole number from 0 to your number. Fullen of 5 is 5 + 4 + 3 + 2 + 1 + 0 = `15`.
- **Adden (mode 2):** adds up the Fullen numbers from 0 to your number. Adden of 5 is 15 + 10 + 6 + 3 + 1 + 0 = `35`.

You type	You get
5, mode 1	`15`
10, mode 1	`55`
5, mode 2	`35`
10, mode 2	`220`

Negative numbers give `0`. A mode other than 1 or 2 ends with an "Unknown mode" error. Answers print as whole numbers, with no `.0`. Adden does far more work than Fullen as the number grows, so very large inputs can take a long time.

Special Operations

Special operations are operations 6-14. These operations are called special because(With the expectation of 12, converter) they would not be found on any other calculator, are more complex, and most don't use "Enter 1st number:" and "Enter 2nd number:".

When something goes wrong

What you see	What happened
`Unknown operator`	The menu choice wasn't 1 to 14.
Error ending in `could not convert string to float`	You typed letters where a number belongs. Restart.
Error ending with the "chucklenuts" joke	You divided by zero in operation 3.

What you see	What happened
`Nice try. Modulus by zero is undefined too.`	You used 0 as the 2nd number in operation 7.
`Positives ONLY`	You entered a negative number in operation 8.
`You are not injecting today, buckaroo!`	The operation 9 modifier had letters or brackets.
`Whoa there, buckaroo!...`	The operation 9 modifier was longer than 10 characters.
`ZeroDivisionError` in the mean	You asked for 0 data points. Enter at least 1.
`Unknown conversion`	The converter mode wasn't 1 to 8.
Unknown mode	The Fullen and Adden mode wasn't 1 or 2.
`Cannot simplify 0:0`	Both numbers in operation 13 were 0.
`OverflowError`	The numbers got too big for Python to store as decimals.
Nothing happens for a long time	A huge factorial, exponent, or Fullen/Adden number is running. Stop the program and restart.

None of these errors change anything on your computer. Restarting the program is always the fix.

Good to know

- **Precision:** PyCalc uses regular decimal numbers, which hold about 15 to 16 digits. Anything past that is rounded when you type it in.
- **Operation 9 uses `eval`:** it runs the modifier you type as Python math. The letter, bracket, and length checks keep that safe for personal use, but don't copy this part into something that other people can type into, like a website or a bot.
- **Output format:** most answers print with a `.0` (like `7.0`) because inputs are read as decimals.

Quick start

1. Run the program.
2. Type `1` for sum and press Enter.
3. Enter `4`, then `3`.
4. Read `7.0`.